

# Dynamic, per-game `Action` / `KeyMap` — what's required

---

*Revised 2026-07-23: updated against the actual code after the `Action` / `KeyMap` shape change, the `ActionSet` design, and the `int id`-keyed `KeyMap` decision were applied. Earlier revisions of this doc described the target shape in the abstract; this revision tracks exactly which files already match it and which don't, so it can be worked top-to-bottom to a compiling, dynamic, per-game input pipeline.*

## Executive summary

---

The core shape decisions are made and partly applied: `Action` is a plain `struct{ id, name, isComposite }` with no fixed-size members, `KeyMap` is `unordered_map<int, vector<KeyCode>>` keyed by `id` (not by `Action` itself — no hashing/equality needed on `Action` for this), and a new `ActionSet` type ( `core/input/action-set.hpp` ) provides `RegisterAction` / `Get` / `All` / `BuildKeyMap` so a game can define its own action set by name instead of a shared enum. `InputConfig` and `InputDispatcher`'s header already reflect the new shape.

What's left is mechanical but touches five files whose `.cpp` /call sites haven't caught up to their own headers yet (some don't compile as-is), plus one open design question ( `ActionSet` vs. the earlier hand-rolled `SnakeInputActions` — pick one), plus the one piece with no home yet: getting a specific action's `id` from `Configure()`'s local scope down to `Snake`, which needs it to know "was this the MoveLeft action" without a shared enum to switch on.

## Where things stand right now

| File                                                        | Status                                                                                                                                                                                                                                                                                                                                                                                                                         |
|-------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>core/input/action.hpp</code>                          | ✅ Done — plain struct, no fixed-size members.                                                                                                                                                                                                                                                                                                                                                                                  |
| <code>core/input/key-map.hpp</code>                         | ✅ Done — <code>unordered_map&lt;int, vector&lt;KeyCode&gt;&gt;</code> .                                                                                                                                                                                                                                                                                                                                                        |
| <code>core/input/action-set.hpp</code>                      | 🟡 Declared, not implemented — <code>RegisterAction</code> , <code>Get</code> , <code>All</code> have no <code>.cpp</code> ; there is no <code>action-set.cpp</code> .                                                                                                                                                                                                                                                          |
| <code>engine/config/input-config.hpp</code>                 | ✅ Done — carries both <code>keyMap</code> and <code>actions</code> .                                                                                                                                                                                                                                                                                                                                                           |
| <code>engine/input/input-dispatcher.hpp</code>              | ✅ Done — constructor takes <code>vector&lt;Action&gt;</code> ; members are <code>vector&lt;Action&gt; actions</code> + two <code>vector&lt;bool&gt;</code> .                                                                                                                                                                                                                                                                   |
| <code>engine/input/input-dispatcher.cpp</code>              | 🔴 Not updated — constructor body still <code>InputDispatcher(eventBus)</code> (drops the <code>actions</code> param entirely, doesn't compile against its own header); <code>Run()</code> still loops <code>0..Action::Count</code> and <code>static_cast&lt;Action&gt;(i)</code> , both invalid now that <code>Action</code> isn't an enum; <code>IsActionDown</code> indexes <code>currentDown[action]</code> with a struct. |
| <code>engine/input/input-system.hpp</code>                  | ✅ Done — constructor takes the extra <code>vector&lt;Action&gt; actions</code> .                                                                                                                                                                                                                                                                                                                                               |
| <code>engine/input/input-system.cpp</code>                  | 🔴 Not updated — constructor definition still takes only 3 params, still builds <code>dispatcher(eventBus)</code> — mismatched against its own header.                                                                                                                                                                                                                                                                          |
| <code>engine/input/sources/keyboard-input-source.cpp</code> | 🔴 Not updated — <code>IsActionDown</code> still does <code>keyMap[static_cast&lt;size_t&gt;(action)]</code> ; <code>KeyMap</code> is keyed by <code>int id</code> , <code>action</code> is a struct, this doesn't compile.                                                                                                                                                                                                     |
| <code>engine/application/application.cpp:80-82</code>       | 🔴 Not updated — constructs <code>InputSystem</code> with only 3 args; needs a 4th ( <code>config.engine.input.actions</code> ) now that the header requires it.                                                                                                                                                                                                                                                                |
| <code>snake-game/snake-application.cpp::Configure()</code>  | 🔴 Not updated — still hand-builds <code>KeyMap</code> as a positional list of <code>{int, vector&lt;KeyCode&gt;}</code> pairs with comments naming the intended action (same fragility as the old enum-ordinal version, just on raw ints); never populates <code>config.engine.input.actions</code> at all, so it's silently empty.                                                                                            |
| <code>snake-game/settings/snake-input-actions.hpp</code>    | 🟡 Superseded — see decision below.                                                                                                                                                                                                                                                                                                                                                                                             |
| <code>snake-game/game-entities/snake.cpp</code>             | 🔴 Not updated — <code>switch (e.action) { case Core::Input::Action::MoveLeft: ... }</code> — <code>Action</code> has no enumerators anymore, doesn't compile.                                                                                                                                                                                                                                                                  |

Nothing above is a design gap at this point — every /  row has a mechanical fix, given below in dependency order (each step compiles cleanly against the previous one).

## Decision: `ActionSet` replaces `SnakeInputActions`

Two competing ways to define "this game's actions" exist in the tree right now: the hand-written `SnakeInputActions` struct (fixed named members, manually assigned `.id`) and the generic `ActionSet` (`RegisterAction` + `Get` + `BuildKeyMap`, auto-incrementing `id`). Keep `ActionSet` and delete `snake-input-actions.hpp` — `ActionSet` removes the manual-id-collision risk `SnakeInputActions` still has, and its `BuildKeyMap` / `Get` give every call site (both `Configure()` and anything later that needs a specific action's `id`, e.g. `Snake`) the same name-based lookup instead of two parallel mechanisms to keep in sync.

That last clause matters: `Get(name)` is only useful if the `ActionSet` instance it was registered on is still alive later. Register actions on a short-lived local inside `Configure()` and `Snake` has nothing to look "MoveLeft" up on afterwards. So the `ActionSet` needs to **outlive** `Configure()`.

**Where it actually landed:** on `Engine::Application` itself (the shared base), not `SnakeApplication` — every game gets an action set for free, not just Snake. `IApplication::GetInputActions() const` is a pure-virtual interface method (`i-application.hpp:47`), so the public/interface-facing accessor has to stay read-only — anything holding an `IApplication&` (systems, scenes) should only ever read the action set, never register into it. `Configure()` (a subclass override, not going through the interface) still needs write access, so `Application` carries a second, non-virtual, `protected` overload distinguished purely by constness — standard const/non-const overloading, resolved automatically by whether `this` is const at the call site:

```
// application.hpp
class Application : public Core::IApplication {
protected:
    ...
    [[nodiscard]] virtual const Core::Input::ActionSet& GetInputActions() const override;
    [[nodiscard]] Core::Input::ActionSet& GetInputActions();    // new – non-const, subclass-
only

private:
    ...
    Core::Input::ActionSet inputActions;
};
```

```
// application.cpp
const Core::Input::ActionSet& Application::GetInputActions() const { return this-
>inputActions; }
Core::Input::ActionSet& Application::GetInputActions() { return this->inputActions; }
```

`inputActions` stays `private` on `Application` — no `protected` data member, consistent with how every other subsystem here (`config`, `injector`, `sceneManager`, `eventBus`, `renderComponentManager`) is private-plus-getter, not a one-off exception for this one field.

```

// snake-application.cpp
void SnakeApplication::Configure(Engine::Config::ApplicationConfig& config)
{
    ...
    this->GetInputActions().RegisterAction("MoveUp");
    this->GetInputActions().RegisterAction("MoveLeft");
    this->GetInputActions().RegisterAction("MoveRight");
    this->GetInputActions().RegisterAction("MoveDown");
    this->GetInputActions().RegisterAction("Confirm");
    this->GetInputActions().RegisterAction("Decline");

    config = {
        .engine = {
            .window = {.width = 500, .height = 500},
            .input = {
                .keyMap = this->GetInputActions().BuildKeyMap({
                    {"MoveUp",    {KeyCode::W, KeyCode::Up}},
                    {"MoveLeft",   {KeyCode::A, KeyCode::Left}},
                    {"MoveRight",  {KeyCode::D, KeyCode::Right}},
                    {"MoveDown",   {KeyCode::S, KeyCode::Down}},
                    {"Confirm",    {KeyCode::Enter}},
                    {"Decline",    {KeyCode::Escape}},
                }),
                .actions = this->GetInputActions().All(),
            },
            .debug = {.enabled = false, .showDebugHud = false, .showDebugLogs = false},
        },
        .project = {.title = "Snake"},
        .game = std::move(snakeSettings),
    };
}

```

This is what actually fixes the silently-empty `config.engine.input.actions` bug noted in the table above — `.actions = this->inputActions.All()` has to be set explicitly; nothing derives it from `keyMap` automatically ( `keyMap` 's keys are bare `int` s, not full `Action` s, by the id-vs-Action keying decision made earlier).

## Remaining work, file by file

---

### 1. `core/input/action-set.cpp` (new file — implement the declared methods)

```
#include "core/input/action-set.hpp"
#include <algorithm>
#include <cassert>

namespace Core::Input {

Action ActionSet::RegisterAction(std::string name)
{
    Action action{.id = this->nextId++, .name = std::move(name)};
    this->actions.push_back(action);
    return action;
}

const Action& ActionSet::Get(const std::string& name) const
{
    auto it = std::find_if(
        this->actions.begin(), this->actions.end(),
        [&name](const Action& a) { return a.name == name; }
    );
    assert(it != this->actions.end() && "ActionSet::Get: unregistered action name");
    return *it;
}

const std::vector<Action>& ActionSet::All() const { return this->actions; }

} // namespace Core::Input
```

`Get` asserts rather than returning a default-constructed `Action` — a typo'd name should fail loudly at startup, not silently resolve to "no keys bound."

## 2. `engine/input/input-dispatcher.cpp` (constructor, `Run`, `IsActionDown`)

```
InputDispatcher::InputDispatcher(Core::Events::IEventBus& eventBus,
std::vector<Core::Input::Action> actions)
    : actions(std::move(actions)), eventBus(eventBus)
{
    this->lastFrameDown.assign(this->actions.size(), false);
    this->currentDown.assign(this->actions.size(), false);
}

void InputDispatcher::Run()
{
    for (size_t i = 0; i < this->actions.size(); ++i) {
        const auto& action = this->actions[i];

        bool isDown = false;
        for (auto& source : this->sources) {
            if (source->IsActionDown(action)) {
                isDown = true;
                break;
            }
        }

        this->currentDown[i] = isDown;
        bool wasDown = this->lastFrameDown[i];

        if (isDown != wasDown) {
            if (isDown) {
                this->eventBus.Publish(Engine::Events::Input::InputActionPressedEvent{action});
            }
            if (wasDown) {
                this->eventBus.Publish(Engine::Events::Input::InputActionReleasedEvent{action});
            }
        }
        this->lastFrameDown[i] = isDown;
    }
}

bool InputDispatcher::IsActionDown(Core::Input::Action action) const
{
    for (size_t i = 0; i < this->actions.size(); ++i) {
        if (this->actions[i].id == action.id) return this->currentDown[i];
    }
    return false;
}
```

This is a straight swap of "loop `0..Count`, cast index to `Action`" for "loop the injected `actions` vector directly" — the event-publish and `lastFrameDown` bookkeeping already present in the file today is correct and unchanged. `IsActionDown` does a linear scan over `this->actions`; with a handful of actions per game that's cheaper and simpler than maintaining a `std::unordered_map<int, size_t>` id → index table, and avoids needing `operator==` / `std::hash` on `Action` anywhere in the input layer.

### 3. `engine/input/input-system.cpp` (constructor, forward actions )

```
InputSystem::InputSystem(
    IInputBackend& inputBackend, Core::Events::IEventBus& eventBus,
    const KeyMap& keyMap, std::vector<Action> actions
)
    : dispatcher(eventBus, std::move(actions))
{
    this->dispatcher.AddSource(
        std::make_unique<Engine::Input::Sources::KeyboardInputSource>(inputBackend, keyMap)
    );
}
```

### 4. `engine/input/sources/keyboard-input-source.cpp` (index by action.id )

```
bool KeyboardInputSource::IsActionDown(Core::Input::Action action) const
{
    auto it = this->keyMap.find(action.id);
    if (it == this->keyMap.end()) return false;
    for (Core::Input::KeyCode keyCode : it->second) {
        if (this->backend.IsKeyDown(keyCode)) return true;
    }
    return false;
}
```

Use `find`, not `operator[]` — `IsActionDown` is `const`, and `unordered_map::operator[]` isn't callable on a `const` map (it inserts a default entry on a missing key, which needs write access). `find` also gives the right behaviour for an action with no bound keys: `keyMap.end()` → not down, rather than inserting an empty vector into the map on every miss.

### 5. `engine/application/application.cpp:80-82` (pass the actions through)

```
auto inputSystem = std::make_shared<Engine::Input::InputSystem>(
    *inputBackend, *eventBus, config.engine.input.keyMap, config.engine.input.actions
);
```

### 6. `engine/application/application.{hpp,cpp}` + `snake-game/snake-application.cpp` — see the `ActionSet` member/getter split and `Configure()` rewrite above.

### 7. Delete `snake-game/settings/snake-input-actions.hpp`

Superseded by `ActionSet` per the decision above — nothing should reference `SnakeInputActions` once `Configure()` is rewritten.

## 8. Thread `ActionBindings` down to `Snake` — not `ActionSet`, not `IInputSystem`

Earlier revisions of this section had `Snake` resolve `ActionSet::Get(name)` itself and hold the resulting `Action`s (and, in the axis draft, an `IInputSystem&`) as members. That's the wrong owner for that job — knowing about `Action`, ids, or the input system's polling API isn't something a game entity should need. `Snake`'s actual job is just "tell input something to call me when `MoveLeft` fires" — which is exactly what `ActionBindings` (§9) is for. `Snake` should depend on `ActionBindings` alone; nothing about `ActionSet` or `IInputSystem` needs to reach `Snake` at all.

```
// gameplay-scene.hpp
struct GameplaySceneParams {
    Core::Events::IEventBus& eventBus;
    Core::Rendering::IRenderComponentManager& renderComponentManager;
    const SnakeGameSettings& gameSettings;
    Core::Input::ActionBindings& inputBindings; // new – replaces inputActions/inputSystem
    int screenWidth;
    int screenHeight;
};
```

```
// snake.hpp
struct SnakeParams {
    Core::Entities::IEntityManager& entityManager;
    const SnakeGameSettings& settings;
    int screenWidth;
    int screenHeight;
    Core::Events::IEventBus& eventBus;
    Core::Input::ActionBindings& inputBindings; // new – replaces inputActions/inputSystem
};
```

Not `const` — `Bind / BindAxis` (§9) mutate `ActionBindings`' internal registration list, so `Snake` needs a mutable reference, the same way it already needs a mutable `eventBus` to call `Subscribe` on.

`Application` needs one more accessor alongside `GetInputActions()`, but this one doesn't need the const/non-const split that `ActionSet` did — the whole point of `ActionBindings` is that outside code is *expected* to mutate it (that's what registering a binding is), so a single accessor returning a mutable reference is correct, matching the existing `GetEventBus() const` pattern already in this class (a `const`-qualified method returning a non-`const` reference):

```
// application.hpp
[[nodiscard]] Core::Input::ActionBindings& GetInputBindings() const;
```

Where this actually lives is answered in §9 — `ActionBindings` isn't built in `Configure()` alongside `ActionSet`, it's built inside `InputSystem`.

## 9. `ActionBindings` — ownership, ticking, and safe unregistration

**Who owns it, and who ticks it.** `ActionBindings` needs two things `ActionSet` never needed: something to poll every frame for axis values, and somewhere to actually *be* polled from. Both point to the same

answer: `InputSystem` is already a `Core::Systems::ISystem` whose `OnUpdate` is already ticked every frame by the systems list ( `application.cpp` 's `RegisterSystem` call), and it already owns an `InputDispatcher` — so `ActionBindings` should live *inside* `InputSystem`, next to `dispatcher`, not on `Application` next to `ActionSet`:

```
// engine/input/input-system.hpp
class InputSystem final : public Core::Input::IInputSystem {
public:
    InputSystem(IInputBackend&, Core::Events::IEventBus&, const KeyMap&,
               std::vector<Action> actions, const ActionSet& actionSet);
    void OnUpdate(const float deltaTime) override;
    Core::Input::ActionBindings& GetBindings() { return this->actionBindings; }

private:
    InputDispatcher dispatcher;
    Core::Input::ActionBindings actionBindings;
};
```

```
// input-system.cpp
InputSystem::InputSystem(IInputBackend& backend, Core::Events::IEventBus& eventBus,
                        const KeyMap& keyMap, std::vector<Action> actions, const ActionSet&
actionSet)
    : dispatcher(eventBus, std::move(actions))
    , actionBindings(eventBus, actionSet, this->dispatcher)
{
    this->dispatcher.AddSource(std::make_unique<KeyboardInputSource>(backend, keyMap));
}

void InputSystem::OnUpdate(const float)
{
    this->dispatcher.Run();           // boolean state + Pressed/Released events, unchanged
    this->actionBindings.Update();    // NEW – polls axis bindings, invokes BindAxis callbacks
}
```

`ActionBindings` takes `const InputDispatcher&`, not `const IInputSystem&` — a deliberate narrowing from the shape sketched earlier in this doc. Since `Snake` now only ever talks to `ActionBindings` (never to `IInputSystem` directly, per §8), `IInputSystem` 's public interface doesn't need widening with a `GetActionValue` method at all — only the concrete `InputDispatcher` (and, beneath it, `IInputSource` / `KeyboardInputSource`) needs one, which is a smaller, more contained change than the four-file interface spread described in an earlier draft of the "Future phase" section below.

`Application::GetInputBindings()` (§8) forwards to the constructed `InputSystem` 's `GetBindings()` — which means `Application` needs to hold the concrete `InputSystem` (not just the `ISystem` interface it registers) as a member, the same way it already holds `window` / `renderer` /etc., rather than only handing it to `RegisterSystem` and discarding the local.

**The Handle problem.** A naive `Bind` / `BindAxis` returning `void` and storing callbacks in `unordered_map<int, function<...>>` keyed by action id has two problems once more than one game entity ever binds, or once scenes get torn down and rebuilt:

1. Two listeners binding the same action silently clobber each other — the second `Bind("Confirm", ...)` call overwrites the first, one-callback- per-id can't hold both.

2. A **Snake** that's destroyed while still registered leaves a dangling callback behind.

`SceneManager::SwitchTo` releases `Transient` scene instances on exit ( `scene-manager.cpp:23-28` , `entry.instance.reset()` ), so navigating away from `Gameplay` destroys `GameplayScene` and its `Snake` . Nothing in a `void` -returning `Bind` ever *removes* the registration — the `std::function` still holds `this` (the destroyed `Snake*` ) in `ActionBindings` ' storage, ready to be invoked the next time a bound key fires, or the next `Update()` tick for an axis binding. That's a live use-after-free once a menu/gameplay scene switch exists, not a theoretical one.

Both are fixed by making registration return a move-only RAI `Handle` that unregisters on destruction, and switching internal storage from "one callback per action id" to a flat list keyed by a unique per-registration id:

```

// core/input/action-bindings.hpp
namespace Core::Input {

class ActionBindings {
public:
    class Handle {
    public:
        Handle() = default;
        Handle(const Handle&) = delete;
        Handle& operator=(const Handle&) = delete;
        Handle(Handle&& other) noexcept { *this = std::move(other); }
        Handle& operator=(Handle&& other) noexcept
        {
            if (this != &other) {
                Release();
                this->owner = other.owner;
                this->id = other.id;
                other.owner = nullptr;
            }
            return *this;
        }
        ~Handle() { Release(); }

    private:
        friend class ActionBindings;
        Handle(ActionBindings* owner, int id) : owner(owner), id(id) {}
        void Release() { if (this->owner) this->owner->Unbind(this->id); this->owner = nullptr; }

        ActionBindings* owner = nullptr;
        int id = 0;
    };

    ActionBindings(Core::Events::IEventBus& eventBus, const ActionSet& actions, const
InputDispatcher& dispatcher);

    [[nodiscard]] Handle Bind(const std::string& name, std::function<void()> callback);
    [[nodiscard]] Handle BindAxis(const std::string& name, std::function<void(ActionValue)>
callback,
                                float threshold = 0.01f);
    void Update();    // ticked from InputSystem::OnUpdate - polls axis bindings

private:
    void Unbind(int handleId);

    struct DigitalBinding { int handleId; int actionId; std::function<void()> callback; };
    struct AxisBinding {
        int handleId;
        Action action;
        std::function<void(ActionValue)> callback;
        ActionValue lastValue;
        float threshold;
    };

    std::vector<DigitalBinding> digitalBindings;
    std::vector<AxisBinding> axisBindings;
    int nextHandleId = 0;
    const ActionSet& actions;

```

```
    const InputDispatcher& dispatcher;  
};  
  
} // namespace Core::Input
```

`[[nodiscard]]` matters here beyond style — a caller that ignores the returned `Handle` gets a temporary that destructs immediately, unregistering the callback right after registering it. That's a real, easy mistake to make; marking it `nodiscard` at least turns it into a compiler warning instead of a silent no-op.

```

// action-bindings.cpp
ActionBindings::ActionBindings(Core::Events::IEventBus& eventBus, const ActionSet& actions,
                                const InputDispatcher& dispatcher)
    : actions(actions), dispatcher(dispatcher)
{
    eventBus.Subscribe<Engine::Events::Input::InputActionPressedEvent>(
        [this](const auto& e) {
            for (auto& binding : this->digitalBindings) {
                if (binding.actionId == e.action.id) binding.callback();
            }
        }
    );
}

ActionBindings::Handle ActionBindings::Bind(const std::string& name, std::function<void()>
callback)
{
    int handleId = this->nextHandleId++;
    this->digitalBindings.push_back({handleId, this->actions.Get(name).id,
std::move(callback)});
    return Handle(this, handleId);
}

ActionBindings::Handle ActionBindings::BindAxis(const std::string& name,
                                                std::function<void(ActionValue)> callback,
float threshold)
{
    int handleId = this->nextHandleId++;
    this->axisBindings.push_back({handleId, this->actions.Get(name), std::move(callback),
ActionValue{}, threshold});
    return Handle(this, handleId);
}

void ActionBindings::Update()
{
    for (auto& binding : this->axisBindings) {
        ActionValue current = this->dispatcher.GetActionValue(binding.action);
        if (Distance(current, binding.lastValue) > binding.threshold) {
            binding.callback(current);
            binding.lastValue = current;
        }
    }
}

void ActionBindings::Unbind(int handleId)
{
    std::erase_if(this->digitalBindings, [&](auto& b) { return b.handleId == handleId; });
    std::erase_if(this->axisBindings, [&](auto& b) { return b.handleId == handleId; });
}

```

Iterating `digitalBindings` on every `InputActionPressedEvent` (rather than an `unordered_map` lookup) is the trade-off for fixing problem 1 above — multiple listeners on one action are now correct instead of silently last-writer-wins; with a handful of bindings per game this is cheap.

`Distance` is one more small `std::visit`-based helper, the same shape as `ValueTypeOf / Accumulate` from the section below ( `bool` : 0 or 1 for equal/different, `float` : `std::abs(a - b)` , `Vector2D` : length of the difference) — for keyboard-only input this isn't filtering analog noise (a key is either down or not, the value never jitters), it's just "don't call the callback again with an unchanged value every single frame." It becomes a literal analog deadzone for free later, once a gamepad `IInputSource` exists, without changing this shape at all.

`Snake` owns its `Handle`s as members — when `Snake` is destroyed, they destruct in reverse declaration order and unregister *before* anything can invoke the now-dangling `this` , closing the lifetime hole above:

```
// snake.hpp
class Snake final {
    ...
private:
    Core::Input::ActionBindings::Handle moveForwardBinding;
    Core::Input::ActionBindings::Handle moveRightBinding;
};
```

```
// snake.cpp
Snake::Snake(const SnakeParams& snakeParams)
    : entityManager(snakeParams.entityManager)
    , settings(snakeParams.settings)
    , screenWidth(snakeParams.screenWidth)
    , screenHeight(snakeParams.screenHeight)
    , moveForwardBinding(snakeParams.inputBindings.BindAxis(
        "MoveForward", [this](Core::Input::ActionValue v) { this->forward = std::get<float>
(v); }))
    , moveRightBinding(snakeParams.inputBindings.BindAxis(
        "MoveRight", [this](Core::Input::ActionValue v) { this->right = std::get<float>(v);
}))
{
}

void Snake::Update(const float deltaTime)
{
    if (this->forward != 0 || this->right != 0) this->SetDirection({this->right, -this-
>forward});
    ...
}
```

( `forward / right` are plain `float` members `Snake` already needs regardless — the callback just keeps them current; `Update()` never touches `ActionBindings` , `ActionSet` , or `IInputSystem` at all.)

**Ordering: is it a problem that `Bind / BindAxis` happen after `Initialize()` , not during `Configure()` ?** No — checked directly rather than assumed. `SceneManager::SwitchTo` ( `scene-manager.cpp:17-36` ) resolves and constructs its target scene **synchronously**, in the same call: `ResolveScene` ( `scene-manager.cpp:102-116` ) does `entry.instance = entry.factory()` in-place, with no queued/pending-scene deferral anywhere in `SceneManager` . So today, when `SnakeApplication::Initialize()` calls `Engine::Application::Initialize()` (constructing `InputSystem` , and thus `ActionBindings` ) and then calls `SwitchTo("Gameplay")` , the `Gameplay` factory

— and therefore `GameplayScene`, and therefore `Snake`'s `Bind / BindAxis` calls — runs immediately afterward, on the same stack, strictly after `ActionBindings` already exists.

This still holds once a `MainMenu` scene is wired up as the actual entry point (its factory registration is currently commented out in `snake-application.cpp`) and `SwitchTo("Gameplay")` only happens later, when the player navigates from the menu — `ActionBindings` lives on `InputSystem` for the app's entire lifetime, so it's still there no matter how much later `Snake` gets constructed. What a `MainMenu / Gameplay` round trip *does* introduce is the `Handle`-motivating problem above: `Transient` teardown on exit destroys `Snake` while its bindings may still be registered — solved by `Snake` owning `Handle`s, not by anything about construction order.

## Future phase: typed, multi-shape action values (bool / float / Vector2D / Vector3D)

Not needed yet — `Snake`'s movement is still four discrete digital actions bound through `ActionBindings`. This is the shape to reach for once an axis input is actually wanted (e.g. collapsing `MoveUp / MoveLeft / MoveRight / MoveDown` into one continuous `Move` action), sketched now so the `KeyMap / Action` design doesn't have to be revisited twice.

**Value type.** Use a `std::variant`, not Unreal's approach of always storing a 3-float vector tagged with a value-type enum. Unreal's engine-wide modifier/trigger pipeline needs every action's value to be processed generically as raw float components, which is what justifies its uniform storage; this codebase has no such generic pipeline — a handful of concrete consumers each expect one specific shape — so a variant's compile-time-checked access ( `std::get<Vector2D>(value)` ) is a better fit than a type that can be silently misread at runtime.

```
// core/input/action-value.hpp
namespace Core::Input {
    using ActionValue = std::variant<bool, float, Core::Math::Vector2D, Core::Math::Vector3D>;
}
```

**Action gains a declared shape.** Something has to say "`Move` is a `Vector2D` action" up front, both so a zero/default value can be constructed each frame before any keys are summed, and so `BuildKeyMap` can reject a binding whose contribution type doesn't match:

```
// action.hpp
enum class ActionValueType { Boolean, Axis1D, Axis2D, Axis3D };

struct Action {
    int id;
    std::string name;
    ActionValueType type = ActionValueType::Boolean;
    bool isComposite = false;
};
```

`ActionSet::RegisterAction` picks up a second parameter: `RegisterAction(std::string name, ActionValueType type = ActionValueType::Boolean)` .

**Binding shape.** `KeyMap` 's value type stops being a flat `vector<KeyCode>` (→ "any of these keys down means this action is down") and becomes a list of key + contribution pairs — each key contributes a typed delta, summed across whichever keys are currently down:

```
// key-map.hpp
struct KeyBinding {
    KeyCode key;
    ActionValue contribution; // must match the owning Action's declared type
};
using KeyMap = std::unordered_map<int, std::vector<KeyBinding>>;
```

**Authoring interface: a small chainable builder, not raw aggregate-init.** The awkward part of "one action, several scale groups" (e.g. `MoveForward` gets `+1` from `W / Up` and `-1` from `S / Down`) is that a flat `{name, vector<KeyBinding>}` pair can't express "here are two differently- scaled key groups for the same action" without nested braces that don't actually flatten in real C++ ( `std::vector` isn't an aggregate, so `{"MoveForward", Axis1D(...), Axis1D(...)}` doesn't compile as one initializer). A tiny chainable builder gets the same readability without relying on brace-elision that doesn't exist:

```
// core/input/action-binding-builder.hpp
namespace Core::Input {

class ActionBindingBuilder {
public:
    explicit ActionBindingBuilder(std::string name) : name(std::move(name)) {}

    ActionBindingBuilder& Digital(std::vector<KeyCode> keys) {
        for (auto key : keys) this->bindings.push_back({key, true});
        return *this;
    }
    ActionBindingBuilder& Axis1D(std::vector<KeyCode> keys, float scale) {
        for (auto key : keys) this->bindings.push_back({key, scale});
        return *this;
    }
    ActionBindingBuilder& Axis2D(std::vector<KeyCode> keys, Core::Math::Vector2D scale) {
        for (auto key : keys) this->bindings.push_back({key, scale});
        return *this;
    }

    const std::string& Name() const { return this->name; }
    const std::vector<KeyBinding>& Bindings() const { return this->bindings; }

private:
    std::string name;
    std::vector<KeyBinding> bindings;
};

inline ActionBindingBuilder Bind(std::string name) { return
ActionBindingBuilder(std::move(name)); }

} // namespace Core::Input
```

Each `.Axis1D(keys, scale)` / `.Digital(keys)` call reads as its own self-documenting binding group — no `/* Action::MoveUp */` -style comments needed to explain what a bare positional entry means, because the method name and its arguments already say it. Chaining `.Axis1D(...).Axis1D(...)` on one `Bind(name)` is exactly "multiple key groups feeding one action," directly, in real C++.

`ActionSet::BuildKeyMap` takes these builders instead of raw pairs, and **validates the declared type against the bindings actually given** — a `Bind("MoveForward").Digital(...)` on an action registered as `Axis1D` is a mismatch that should fail loudly at `Configure()` time, not silently produce a `KeyBinding` whose `contribution` variant holds the wrong alternative for later `std::get<float>` calls to blow up on:

```

Core::Input::KeyMap ActionSet::BuildKeyMap(std::vector<ActionBindingBuilder> entries) const
{
    Core::Input::KeyMap keyMap;
    for (auto& entry : entries) {
        const Action& action = this->Get(entry.Name());
        assert(std::ranges::all_of(entry.Bindings(), [&](auto& b) {
            return ValueTypeOf(b.contribution) == action.type;
        }) && "ActionBindingBuilder: contribution type doesn't match RegisterAction's declared
type");
        keyMap[action.id] = entry.Bindings();
    }
    return keyMap;
}

```

Put together, `Configure()` reads close to the shape you sketched, minus the positional comments — the builder chain *is* the documentation:

```

// snake-application.cpp – Configure(), once axis input is wanted
this->GetInputActions().RegisterAction("MoveForward", Core::Input::ActionValueType::Axis1D);
this->GetInputActions().RegisterAction("MoveRight",   Core::Input::ActionValueType::Axis1D);
this->GetInputActions().RegisterAction("Confirm",     Core::Input::ActionValueType::Boolean);
this->GetInputActions().RegisterAction("Decline",     Core::Input::ActionValueType::Boolean);

config.engine.input.keyMap = this->GetInputActions().BuildKeyMap({
    Core::Input::Bind("MoveForward")
        .Axis1D({KeyCode::W, KeyCode::Up}, 1.0f)
        .Axis1D({KeyCode::S, KeyCode::Down}, -1.0f),
    Core::Input::Bind("MoveRight")
        .Axis1D({KeyCode::D, KeyCode::Right}, 1.0f)
        .Axis1D({KeyCode::A, KeyCode::Left}, -1.0f),
    Core::Input::Bind("Confirm").Digital({KeyCode::Enter}),
    Core::Input::Bind("Decline").Digital({KeyCode::Escape}),
});
config.engine.input.actions = this->GetInputActions().All();

```

Two named `Axis1D` actions ( `MoveForward` / `MoveRight` ) rather than one `Axis2D` `Move` action, matching classic engine "axis mapping" conventions (pre-Enhanced-Input Unreal works exactly this way — separate named axes, each fed by opposite-signed key groups, combined into a vector by whatever game code reads them) — `Snake::Update()` would read both and build its own `Vector2D` from the two floats, rather than the single `Vector2D`-typed action shown in the previous revision of this section. Either shape (one `Axis2D` action, or two `Axis1D` actions) is representable with the same `ActionValue` / `KeyBinding` machinery — this is a call-site choice, not a structural one.

`Digital` deliberately stays an explicit builder method (not a bare `vector<KeyCode>` shorthand) so every binding group is named consistently with `ActionValueType` regardless of shape — `RegisterAction(name, ActionValueType::Boolean)` next to `Bind(name).Digital(keys)` keeps the "what kind of value is this" story the same everywhere, rather than booleans being the one shape that looks structurally different from the others.

Two consumption models, not one — but both go through `ActionBindings`. Digital actions ( `Confirm` / `Decline` ) stay exactly as they are — edge- triggered `InputActionPressedEvent` / `ActionBindings::Bind` callbacks. Axis actions ( `MoveForward` / `MoveRight` ) don't have a meaningful "press/release" edge in the same sense (the value is continuous and can sit at any point between frames), so they're **polled**, not delegated — but per §8/§9, that polling is `ActionBindings::Update()` 's job internally, not something `Snake` (or any other game entity) ever does itself via a raw `IInputSystem` reference. `Snake` only ever sees `ActionBindings::BindAxis`, already shown in §9 with its `Handle` /deadzone design — nothing further is needed at the `Snake` call site beyond what's there.

What *is* still needed underneath, for `ActionBindings::Update()` to have something real to poll, is a `GetActionValue` method on `InputDispatcher` (and, below it, `IInputSource` / `KeyboardInputSource` — `InputDispatcher` doesn't compute anything itself, only fans out to `sources`):

```
// core/input/i-input-source.hpp
class IInputSource {
public:
    virtual ~IInputSource() = default;
    virtual bool IsActionDown(Action action) const = 0;
    virtual ActionValue GetActionValue(Action action) const = 0;
};
```

```
// engine/input/input-dispatcher.hpp / .cpp
ActionValue InputDispatcher::GetActionValue(Action action) const
{
    // combine across sources the same way IsActionDown does — first non-zero wins,
    // or sum if more than one source is ever expected to contribute simultaneously
    for (auto& source : this->sources) {
        return source->GetActionValue(action);    // single-source (keyboard-only) case
    }
    return {};
}
```

```
// engine/input/sources/keyboard-input-source.cpp
ActionValue KeyboardInputSource::GetActionValue(Action action) const
{
    auto it = this->keyMap.find(action.id);
    if (it == this->keyMap.end()) return {};

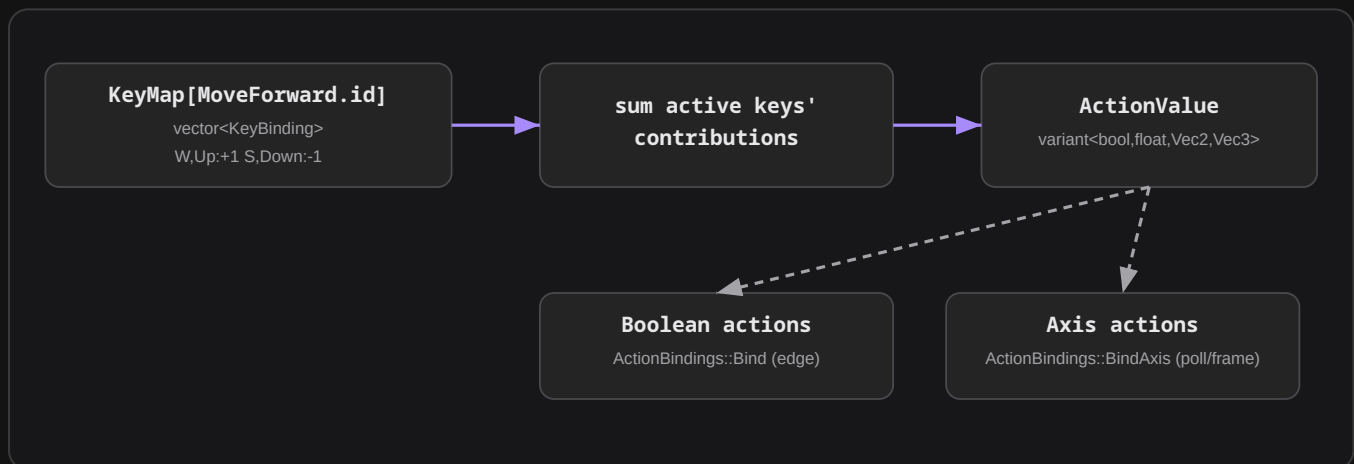
    ActionValue acc{};    // zero-valued for whatever alternative the first binding uses
    for (const KeyBinding& binding : it->second) {
        if (this->backend.IsKeyDown(binding.key)) acc = Accumulate(acc, binding.contribution);
    }
    return acc;
}
```

`Accumulate` is the other piece with no home yet — a small `std::visit`-based function that adds two `ActionValue`s of the same alternative ( `bool` → OR, `float` → `+`, `Vector2D` → component-wise `+` ), the same shape as `ValueTypeOf` above and `Distance` in §9.

Note what's *not* on this list, compared to an earlier draft of this section: `IInputSystem` itself doesn't need `GetActionValue` added to its public interface at all. That earlier draft had `Snake` calling `this->inputSystem.GetActionValue(...)` directly, which would have forced `GetActionValue` onto `IInputSystem` too. Per §8/§9, `Snake` never holds an `IInputSystem` reference — only `ActionBindings` does (as a concrete `InputDispatcher&`, constructed alongside it inside `InputSystem`) — so the interface surface this actually touches is `IInputSource` + `InputDispatcher`, not `IInputSystem`. Narrower than previously scoped, and a direct consequence of keeping `Snake` off `ActionSet` / `IInputSystem` entirely.

This mirrors how Unreal itself splits the two — button-style Input Actions are consumed via bound delegates (`Triggered` / `Started` / `Completed`), axis Input Actions are typically read via `GetActionValue` each tick — except here both paths are fronted by the same `ActionBindings` facade rather than game code choosing between two different subsystems to talk to.

**What this touches, when it's actually built:** `Action` (+type field), `KeyMap`'s value type, `ActionSet::RegisterAction` / `BuildKeyMap`, the new `ActionBindingBuilder`, `Accumulate` / `Distance` helpers next to `ValueTypeOf`, a new `GetActionValue` on `IInputSource` / `KeyboardInputSource` and `InputDispatcher` (not `IInputSystem`), and `ActionBindings::BindAxis` / `Update()` from §9. `ActionBindings::Bind` and its digital dispatch are unaffected — they keep serving `Confirm` / `Decline` exactly as designed.



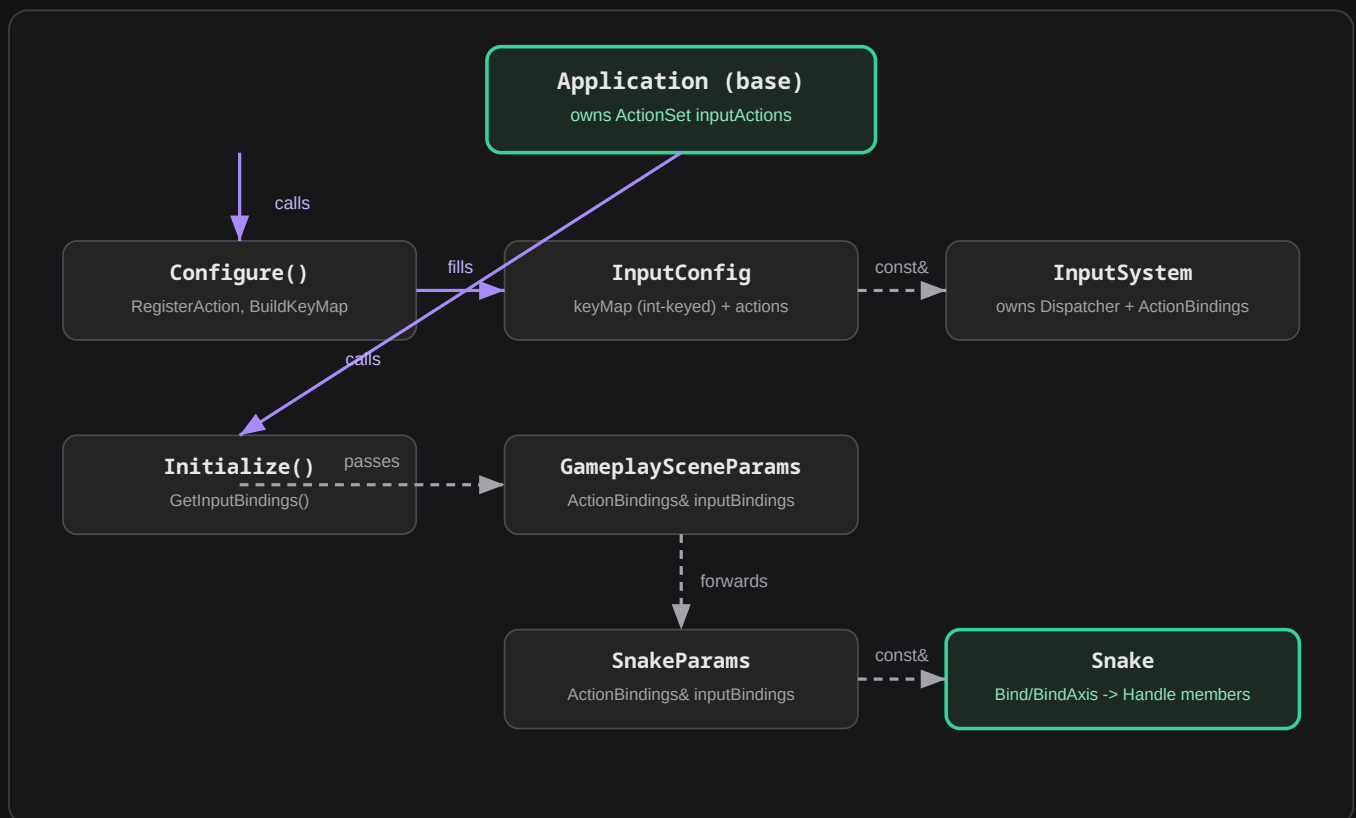
## Migration order (do these in sequence — each compiles against the last)

1. Implement `action-set.cpp` (§1). Nothing depends on anything else yet.
2. Rewrite `input-dispatcher.cpp` (§2) to match its already-updated header.
3. Rewrite `input-system.cpp` (§3) to match its already-updated header.
4. Fix `keyboard-input-source.cpp` (§4).
5. Update the `InputSystem` construction call in `application.cpp` (§5) — the engine-side input pipeline compiles end-to-end at this point, independent of any specific game.
6. Add the `ActionSet` member + const/non-const getter split to `Application`, rewrite `SnakeApplication::Configure()` to register actions and build `keyMap` / `actions` from it (§6). Delete `snake-input-actions.hpp` (§7).

7. Implement `ActionBindings` (§9) — the `Handle`-returning `Bind`, owned by and constructed inside `InputSystem` alongside `dispatcher`, ticked from `InputSystem::OnUpdate`. Add `Application::GetInputBindings()`.
8. Add `inputBindings` (an `ActionBindings&`, **not** `inputActions` / `inputSystem`) to `GameplaySceneParams` and `SnakeParams` (§8); wire `SnakeApplication::Initialize()` to pass `GetInputBindings()` through; rewrite `Snake`'s constructor to `Bind(...)` calls returning `Handle` members, instead of manual `e.action.id` comparisons.

Steps 1–5 are engine-layer and game-agnostic; steps 6–8 are the "per game" half — a second game would only ever touch its own equivalent of steps 6–8, never 1–5. The typed-value/axis-binding phase above (including `BindAxis` /deadzone/ `GetActionValue` on `InputDispatcher`) is future work, not part of this migration — it only matters once an axis-shaped action is actually needed. The `Handle` /RAII unregistration design in §9, however, is **not** future work — it's needed as soon as any scene other than `Gameplay` exists (e.g. `MainMenu`), since that's what makes `Transient` scene teardown safe.

## Ownership/lifetime, current target shape



■ single, persistent owner of the game's action definitions ■ non-owning `const&`, threaded through construction params

## Summary table

| Requirement                                                                                                                      | Resolved by                                                                                                                                                                                                                           |
|----------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| No shared/fixed action enum                                                                                                      | <code>ActionSet::RegisterAction</code> — each game builds its own set by name                                                                                                                                                         |
| <code>KeyMap</code> dynamically sized, no hashing<br><code>Action</code>                                                         | Already done — <code>unordered_map&lt;int, vector&lt;KeyCode&gt;&gt;</code> , <code>int id</code> as key                                                                                                                              |
| Config-time UX (name-based, no held variables)                                                                                   | <code>ActionSet::BuildKeyMap</code> — resolves names to ids internally                                                                                                                                                                |
| <code>InputDispatcher</code> / <code>InputSystem</code> no longer assume <code>Action::Count</code>                              | §2–§3 rewrite to iterate the injected <code>vector&lt;Action&gt;</code>                                                                                                                                                               |
| Game code ( <code>Snake</code> ) can register per-action behavior without an enum or manual id matching                          | §9 — <code>ActionBindings::Bind(name, callback)</code>                                                                                                                                                                                |
| <code>Snake</code> doesn't need to know about <code>ActionSet</code> , <code>Action</code> , or <code>IInputSystem</code> at all | §8 — <code>SnakeParams</code> / <code>GameplaySceneParams</code> carry only <code>ActionBindings&amp;</code> ; <code>ActionBindings</code> is the sole thing holding <code>ActionSet</code> / <code>InputDispatcher</code> references |
| Multiple listeners can bind the same action without clobbering each other                                                        | §9 — flat <code>vector&lt;...Binding&gt;</code> keyed by a unique handle id, not <code>unordered_map&lt;actionId, callback&gt;</code>                                                                                                 |
| A destroyed <code>Snake</code> can't leave a dangling callback in the input system                                               | §9 — <code>Bind</code> / <code>BindAxis</code> return a move-only RAII <code>Handle</code> that unregisters on destruction; verified necessary once <code>Transient</code> scenes ( <code>scene-manager.cpp:23-28</code> ) exist      |
| One mechanism, not two                                                                                                           | Delete <code>snake-input-actions.hpp</code> ; <code>ActionSet</code> is the only per-game action definition path                                                                                                                      |
| Room to grow into axis/vector actions later without redesigning <code>KeyMap</code> twice                                        | Future phase — <code>ActionValue</code> variant + <code>KeyBinding{key, contribution}</code> + <code>ActionBindings::BindAxis</code> with deadzone, sketched but not yet needed                                                       |